

Trabajo Práctico Integrador: Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica .

Asignatura: Desarrollo de Software

Modalidad: en grupo

Criterio de Evaluación y Política de IA: Se permite el uso de herramientas de Inteligencia Artificial (LLMs). Sin embargo, el práctico está diseñado para evaluar el razonamiento crítico, el análisis de estados de memoria y la justificación técnica. Las respuestas genéricas generadas por IA sin prueba de trazabilidad manual o sin responder a los escenarios de borde específicos serán penalizadas.

Criterios Generales de Entrega

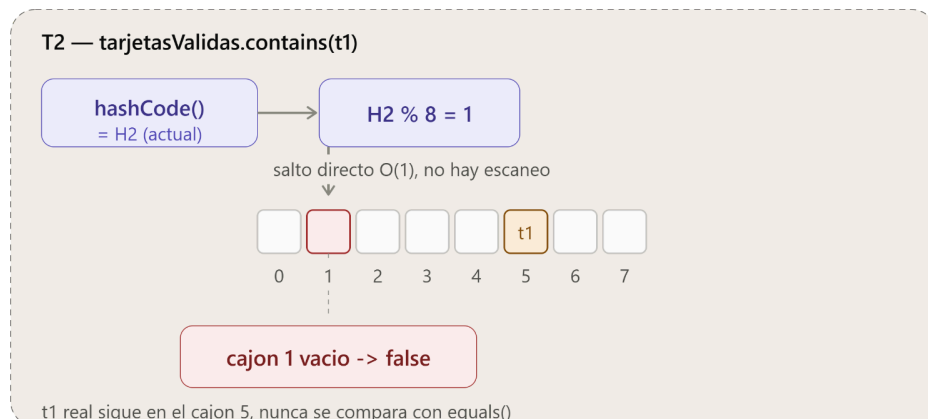
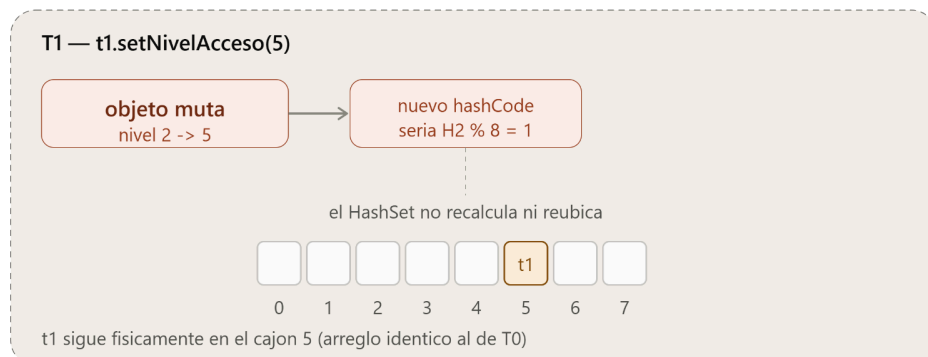
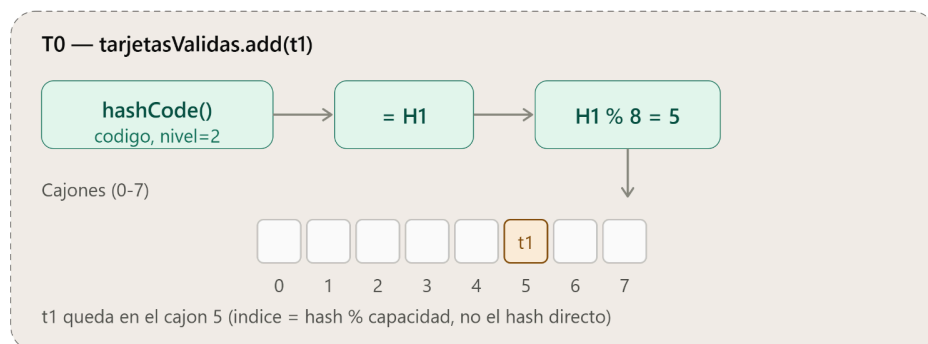
- Entregar un informe en formato PDF publicado en un repositorio de GitHub.
- Incluir fragmentos de código fuente en Java y los diagramas/trazas explicativas solicitadas.

Integrantes:

- Day pi, Nicolas
- Echegaray, Juan Pablo
- Escoriaza, Santino
- Friscolanti, Valentino
- Lopez, Lionel
- Salinas, Julian

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

- 1. Traza del Estado Interno:** Al insertar la tarjeta mediante `tarjetasValidas.add(t1)`, Java evalúa su `hashCode` y ubica el objeto en un cajón específico. Cuando ejecutas `t1.setNivelAcceso(5)`, el estado del objeto muta y su hash cambia (*debido a que el hash depende del código y el nivel de acceso*), pero el `HashSet` no reordena sus elementos automáticamente. Finalmente, al llamar a `tarjetasValidas.contains(t1)`, Java calcula el nuevo hash y busca en un cajón completamente distinto. El resultado es falso porque el objeto original quedó físicamente atrapado en el primer cajón (el que tenía antes de hacer set).
- 2. Explicación del Cajón:** Básicamente el objeto quedó guardado en un primer cajón (n°5 por ejemplo) en el momento en que se insertó mediante `tarjetasValidas.add(t1)`. Luego al hacer `contains()`, Java calcula el hash y termina buscando en otro cajón (n°9 ahora), ya que previo a esta instrucción se realizó un set a uno de los atributos que componen el hash haciendo que el nuevo hash sea distinto del primero donde se guardó físicamente el objeto.



3. **Fuga de Memoria (Memory Leak):** En un servidor 24/7, esto genera un *memory leak*. Los objetos mutados quedan "perdidos" en cajones incorrectos; no pueden ser encontrados ni eliminados con métodos como `remove()`. Estas referencias persistentes impiden que el *Garbage Collector* libere esa memoria.
4. **Refactorización Obligatoria:** Tienes dos caminos. El primero es aplicar inmutabilidad: eliminar el método `setNivelAcceso`, hacer las variables `final` y requerir la creación de un nuevo objeto para cualquier cambio. El segundo es ajustar el criterio de entidad: modificar los métodos `equals` y `hashCode` para que solo dependan del campo `codigo` (la identidad real) y excluir `nivelAcceso`.

Aplicando Inmutabilidad

```
import java.util.*;

final class TarjetaAcceso {
    private final String codigo;
    private final int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public String getCodigo() {
        return codigo;
    }

    public int getNivelAcceso() {
        return nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso &&
Objects.equals(codigo, tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {
```

```

    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();
        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);
        tarjetasValidas.add(t1);

        // Para cualquier cambio se requiere crear un nuevo objeto:
        tarjetasValidas.remove(t1);
        TarjetaAcceso t1Modificada = new TarjetaAcceso(t1.getCodigo(),
5);
        tarjetasValidas.add(t1Modificada);

        System.out.println("¿Tarjeta válida en sistema?: " +
tarjetasValidas.contains(t1Modificada));
        System.out.println("Cantidad de tarjetas registradas: " +
tarjetasValidas.size());
    }
}

```

Aplicando el criterio de entidad

```

import java.util.*;
class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;
    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }
    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return
false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return Objects.equals(codigo, tarjeta.codigo); //SOLO SE
USA CODIGO PARA EL EQUALS
    }
    @Override
    public int hashCode() {
        return Objects.hash(codigo); //SOLO SE USA CODIGO PARA
CALCULAR
    }
}

```

```

public class ControlAcceso {
    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();
        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);
        tarjetasValidas.add(t1);
        // Modificación posterior a la inserción
        t1.setNivelAcceso(5);
        System.out.println("¿Tarjeta válida en sistema?: " +
            tarjetasValidas.contains(t1));
        System.out.println("Cantidad de tarjetas registradas: " +
            tarjetasValidas.size());
    }
}

```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Requerimiento	Interfaz	Implementación	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica
1. Auditoría PDF	List	ArrayList	$O(n)$	$O(1)$ amortizado	Inserción ultrarrápida al final. Su almacenamiento contiguo en memoria favorece el recorrido secuencial.
2. Ofertas PDF	Set	TreeSet	$O(\log n)$	$O(\log n)$	Implementa un árbol rojinegro que mantiene los elementos ordenados automáticamente por precio de menor a mayor.
3. Sesiones PDF	Map	HashMap	$O(1)$	$O(1)$	Acceso instantáneo mediante una función de dispersión (<i>hash</i>) utilizando el Token como clave única.

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

1. **Refactorización a $O(n)$:** Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).

```
public static Character primerCaracterNoRepetidoEficiente(String texto) {
    Map<Character, Integer> mapa = new LinkedHashMap<>();
    for (char c : texto.toCharArray()) {
        mapa.put(c, mapa.getOrDefault(c, 0) + 1);
    }
    for (Map.Entry<Character, Integer> entry : mapa.entrySet()) {
        if (entry.getValue() == 1) return entry.getKey();
    }
    return null;
}
```

2. **Análisis de Colisiones:** Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Si dos caracteres generan un mismo hashCode, el HashMap no sobrescribe el dato. Almacena ambos elementos en el mismo cajón utilizando internamente una lista enlazada (o un árbol binario) y utiliza el método equals() al buscar para devolver el valor exacto.

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que `s1.hashCode() == s2.hashCode()` devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un HashSet, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos hashCode() y equals() internamente para determinar si s2 debe ingresarse o desecharse.

Si insertas "FB" y "Ea", el HashSet contendrá 2 elementos. Aunque `s1.hashCode() == s2.hashCode()` devuelve true, Java siempre ejecuta `s1.equals(s2)` como validación final. Al ser cadenas lógicamente diferentes, el resultado es false y Java almacena ambas en el mismo cajón.

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un HashMap en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

En el peor escenario de colisiones continuas, todos los elementos caerían en un solo cajón perdiendo la ventaja de acceder a los elementos en un solo paso. Antiguamente esto degradaba el rendimiento de $O(1)$ a $O(n)$ (lista enlazada). Sin embargo, desde Java 8, si un cajón acumula más de 8 elementos, la estructura muta a un árbol binario balanceado, limitando el peor escenario a $O(\log n)$.